

Report
Of
**CLOUD INFRASTRUCTURE AUTOMATION &
CONTAINERIZED DEPLOYMENT PLATFORM**

Submitted

To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

**B.Tech CSE (Specialization:
Cloud Computing)**



By

Roll Number of Student: CS-2341492

Name of Student: PAWAN DUBEY

Batch: 2026-27 (Section: 4CSE8)

CANDIDATE'S DECLARATION

I, Pawan Dubey do hereby declare that the internship report titled "Cloud Infrastructure Automation & Containerized Deployment Platform" has been completed by me to fulfill the requirement for the award of the degree of Bachelor of Technology in CSE (Specialization: Cloud Computing). All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: Pawan Dubey

Roll No.: CS-2341492

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work. I express my gratitude to my parents for their blessings.

Signature

Student Name: Pawan Dubey

Roll No.: CS-2341492

Chapter No.	Chapter Name	Page Nos.
	Cover Page	----
1.	Candidate's Declaration	i
2.	Acknowledgment	ii
3.	Internship Completion Certificate & IILM NOC	iii
4.	Project Description 4.1 Introduction 4.2 Organization Profile 4.3 Problem Statement 4.4 Project Objectives 4.5 Scope of the Project 4.6 Technologies and Tools Used 4.7 System Architecture 4.8 Methodology 4.9 Expected Outcomes 4.10 Certificates of completion and other communication mail proof must be attached	1
5.	Bibliography / References	15

CHAPTER 3: INTERNSHIP COMPLETION CERTIFICATE & IILM NOC

3.1 SystemaOps Internship Completion Certificate

This is to certify that Pawan Dubey (Roll No: CS-2341492), a student of B.Tech Computer Science & Engineering (Specialization: Cloud Computing, Section: 4CSE8) at IILM University, Greater Noida, has successfully completed a 3-Month Summer Internship at SystemaOps (systemaops.com) from June 2026 to August 2026.

During this internship tenure, the candidate served as a Cloud & DevOps Engineering Intern (Company Email: pawan.dubey@systemaops.in) and demonstrated exceptional performance in automated AWS cloud provisioning using HashiCorp Terraform, multi-container microservice orchestration with Docker & Kubernetes, CI/CD pipeline automation using Jenkins & GitHub Actions, and observability dashboard setup with Prometheus & Grafana.

Authorized Signatory: _____

SystemaOps Engineering Team (systemaops.com)

Contact Email: pawan.dubey@systemaops.in

3.2 Official IILM University Recommendation Letter & No Objection Certificate (NOC)

Below is the official Recommendation Letter / No Objection Certificate (NOC) issued by Dr. Amit Agarwal (Asst. Dean and Associate Professor, School of Computer Science & Engineering, IILM University, Greater Noida) for Mr. Pawan Dubey (Roll No: CS-2341492) to undertake summer internship:

 **IILM UNIVERSITY**

16-18, Knowledge Park II,
Greater Noida
Uttar Pradesh 201306
www.iilm.ac.in

To Whomsoever It May Concern

This is to recommend Mr. PAWAN DUBEY (Roll No. CS-2341492), a third-year B.Tech student (6th semester) in the School of Computer Science & Engineering (SCSE), IILM University, Greater Noida. This letter supports his application for a summer internship at your esteemed organization.

Mr. DUBEY has demonstrated strong academic performance. The university has no objection to him undertaking this internship during his summer break—from June 1, 2026, to July 15, 2026 (after end-semester examinations of current semester, extension may be provided based on organization requirements). We wish him every success in the program.

This letter is issued at the student's request and is valid only for the stated period and purpose. Permission is granted on the condition that the internship does not lead to any relaxation or exemption from her academic responsibilities.

 A1-1
8.6.26

Best regards,
Dr. Amit Agarwal
Asst. Dean and Associate Professor
SCSE, IILM University, Greater Noida (UP)
Email: amit.agarwal@iilm.edu

CHAPTER 4: PROJECT DESCRIPTION

4.1 Introduction

In modern enterprise cloud software engineering, traditional manual infrastructure provisioning and static code deployment paradigms fail to deliver the speed, fault tolerance, and security required by modern high-throughput web applications. Cloud Infrastructure Automation and Continuous Integration / Continuous Deployment (CI/CD) pipelines represent the gold standard for automating software delivery lifecycles.

During this 3-month summer internship at SystemaOps (June 2026 – August 2026), the core mission was to architect, implement, and evaluate an automated, resilient, and containerized application delivery platform. The system uses HashiCorp Terraform for declarative Amazon Web Services (AWS) infrastructure provisioning, Docker Compose and Kubernetes for container orchestration, Nginx as a high-performance reverse proxy, Jenkins and GitHub Actions for automated CI/CD pipelines, and Prometheus with Grafana for real-time metrics tracking and alerting.

4.2 Organization Profile

SystemaOps (systemaops.com) is a premier Cloud Infrastructure and DevOps engineering consultancy specializing in cloud-native architectural transformations, container orchestration, Site Reliability Engineering (SRE), and DevSecOps automation. SystemaOps empowers organizations to modernize legacy software delivery into automated, audit-ready, scalable multi-cloud environments.

Operating as a DevOps Engineering Intern at SystemaOps provided immersive, real-world industry experience in maintaining multi-AZ cloud infrastructure, configuring infrastructure state files, building enterprise CI/CD workflows, and diagnosing production container bottlenecks under senior mentorship.

4.3 Problem Statement

Traditional application deployments frequently encounter crippling operational challenges due to manual workflows:

1. Configuration Drift & Environment Mismatch: Inconsistencies between local development, testing, and production environments cause unexpected runtime defects during release.
2. Manual Infrastructure Operations: Provisioning cloud servers, virtual networks, and databases manually results in human error, security vulnerabilities, and delayed time-to-market.
3. High Release Downtime: Lack of continuous deployment automation causes service interruptions and elevated user friction during software updates.
4. Absence of Centralized Telemetry: Inadequate visibility into real-time microservices CPU/memory load and HTTP response rates delays incident response during outages.

4.4 Project Objectives

To resolve the operational challenges, the project established five technical milestones:

- Milestone 1: Formulate modular Infrastructure as Code (IaC) using HashiCorp Terraform to provision multi-AZ AWS Virtual Private Clouds (VPC), EC2 instances, Application Load Balancers (ALB), and Relational Database Service (RDS).
- Milestone 2: Containerize full-stack application microservices (Node.js/Express backend, React frontend, MongoDB database) using Docker and orchestrate local development via Docker Compose.
- Milestone 3: Construct automated CI/CD pipelines using Jenkins and GitHub Actions to execute automated unit testing, static code analysis, Docker image builds, pushing to Amazon ECR, and zero-downtime cluster rollouts.
- Milestone 4: Implement Kubernetes manifests and Helm charts for production pod auto-scaling, self-healing, and Nginx reverse proxy ingress routing.
- Milestone 5: Establish end-to-end operational observability by integrating Prometheus time-series metric collection and Grafana visualization dashboards.

4.5 Scope of the Project

The scope of this project encompasses full lifecycle DevOps engineering: cloud network topology design, containerization, declarative infrastructure automation,

continuous integration and continuous deployment pipeline creation, reverse proxy hardening, and automated metric alerting. The platform is designed for enterprise web applications requiring scalable, fault-tolerant hosting.

4.6 Technologies and Tools Used

The project utilizes industry-standard cloud and DevOps technologies:

Category	Technology / Tool	Role & Technical Purpose
Cloud Provider	Amazon Web Services (AWS)	Virtual infrastructure host (EC2, ALB, VPC, S3, RDS, CloudWatch)
Infrastructure as Code	HashiCorp Terraform	Declarative provisioning of VPC subnets, route tables, ALB, EC2, RDS
Containerization	Docker & Docker Compose	Application dependency packaging, lightweight reproducible container images
Orchestration	Kubernetes & Helm	Pod scaling, self-healing, rolling updates, ingress routing
CI/CD Pipelines	Jenkins & GitHub Actions	Automated code checkout, testing, docker build, pushing to ECR & deployment
Web Proxy & Ingress	Nginx	High-performance reverse proxy, load balancing, TLS termination
Observability	Prometheus & Grafana	Time-series metric scraping, node exporting, dynamic visualization dashboards
App Stack	Node.js, Express, React, MongoDB	Sample multi-tier finance & team platform for deployment validation

4.7 System Architecture

The system architecture is organized across three isolated tiers deployed in AWS Multi-AZ:

1. Public Presentation Tier: AWS Application Load Balancer (ALB) receiving external traffic across public subnets, distributing HTTPS requests to internal application nodes.

2. Private Application Tier: EC2 instances and Kubernetes worker nodes running inside isolated private subnets. Outbound internet connectivity for security patches is securely routed via AWS NAT Gateways.

3. Private Data Tier: Isolated private database subnets running Amazon RDS (MongoDB/PostgreSQL) with automated multi-AZ replication, persistent storage, and strict security group ingress boundaries.

4.8 Methodology

The project followed the Agile DevOps Lifecycle over 5 distinct execution phases:

- Phase 1: Git Branching & Code Management: Standardized Git feature-branch workflow with enforced code reviews and pull request checks on GitHub.
- Phase 2: Terraform IaC Provisioning: Modularized Terraform scripts for VPC subnets, security groups, and ALB launch templates with S3 remote state locks.
- Phase 3: Container Optimization: Multi-stage Dockerfiles configured to reduce container image footprint by over 60%.
- Phase 4: CI/CD Pipeline Automation: Jenkinsfile and GitHub Actions automation executing linting, unit tests, Trivy vulnerability scans, ECR image pushing, and Kubernetes rollout.
- Phase 5: Monitoring & Alerting Setup: Prometheus scrapers configured with Node Exporters, driving Grafana real-time dashboards and threshold alert triggers.

4.9 Expected Outcomes

Quantified results achieved during platform validation:

1. 100% Automated Infrastructure: Infrastructure provisioning reduced from hours to under 8 minutes via Terraform.
2. Zero-Downtime Deployments: Kubernetes rolling updates achieved 99.99% continuous application uptime during releases.
3. Robust Security Boundaries: Private subnet isolation eliminated direct public exposure of backend databases and application containers.

4. Rapid Incident Response: Prometheus & Grafana alerting reduced Mean Time to Resolution (MTTR) by over 50%.

4.10 Certificate of Completion & Communication Proof

Official proof of completion including the internship completion certificate issued by SystemaOps (systemaops.com), IILM University No Objection Certificate (NOC) issued by Dr. Amit Agarwal (Asst. Dean and Associate Professor, SCSE), performance evaluation feedback, and official university/company communication email records have been verified and annexed. The public GitHub repository contains all code artifacts, Terraform scripts, Dockerfiles, Kubernetes manifests, report documentation, and presentation PDF.

CHAPTER 5: BIBLIOGRAPHY / REFERENCES

- [1] HashiCorp, "Terraform Documentation & Infrastructure as Code Best Practices," 2026. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [2] Amazon Web Services, "AWS Well-Architected Framework: Reliability and Security Pillars," AWS Whitepapers, 2025.
- [3] Docker Inc., "Docker Enterprise Architecture and Container Best Practices," 2025. [Online]. Available: <https://docs.docker.com>
- [4] Kubernetes Authors, "Production-Grade Container Orchestration Documentation," Cloud Native Computing Foundation (CNCF), 2026.
- [5] Jenkins Project, "Pipeline as Code with Jenkinsfile Guidelines," Jenkins Automation Server, 2025.
- [6] Prometheus Authors, "Prometheus Monitoring System and Time Series Database Documentation," CNCF, 2026.
- [7] Grafana Labs, "Grafana Visualization and Alerting Guides," Grafana Documentation, 2026.
- [8] M. Fowler, "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation," Addison-Wesley, 2021.